

Middle of Linked List

Problem:

Find the middle node of a linked list.

Problem Statement

Given the head of a singly linked list, return the middle node.

If there are two middle nodes, return the second middle node.

Example 1

Input:

10 → 20 → 30 → 40 → 50

Output:

30

Explanation:

- Total nodes = 5
 - Middle node = 3rd node
-

Example 2

Input:

10 → 20 → 30 → 40 → 50 → 60

Output:

40

Explanation:

- Total nodes = 6
 - Two middles = 30 and 40
 - Return second middle = 40
-

Approach 1 — Brute Force

Intuition

To find middle:

1. Count total nodes
 2. Middle position = $n / 2$
 3. Traverse again to middle
-

Thought Process

Suppose:

10 → 20 → 30 → 40 → 50

Count nodes:

$n = 5$

Middle index:

$n / 2 = 2$

Move 2 steps from head:

10 → 20 → 30

Answer = 30

Algorithm

Step 1

Count total nodes

Step 2

Find middle position

middle = $n / 2$

Step 3

Traverse again till middle

Time Complexity

Operation	Complexity
------------------	-------------------

Counting	$O(N)$
----------	--------

Traversing again	$O(N)$
------------------	--------

Total	$O(N)$
-------	--------

Space Complexity

$O(1)$

Dry Run (Brute Force)

Input:

10 → 20 → 30 → 40 → 50

First Traversal

Count nodes:

Node Count

10 1

20 2

30 3

40 4

50 5

$n = 5$

Middle Calculation

middle = $n / 2$

 = $5 / 2$

 = 2

Second Traversal

Start from head:

Step Node

0 10

1 20

2 30

Answer:

30

C++ Code (Brute Force)

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = NULL;
    }
};

Node* middleNode(Node* head) {

    int count = 0;

    Node* temp = head;

    // Count nodes
    while(temp != NULL) {
        count++;
        temp = temp->next;
    }

    int middle = count / 2;

    temp = head;

    // Move to middle
    for(int i = 0; i < middle; i++) {
        temp = temp->next;
    }

    return temp;
}

int main() {

    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);
    head->next->next->next = new Node(40);
    head->next->next->next->next = new Node(50);
```

```
Node* ans = middleNode(head);

cout << ans->data;

return 0;
}
```

Java Code (Brute Force)

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class Main {

    static Node middleNode(Node head) {

        int count = 0;

        Node temp = head;

        // Count nodes
        while(temp != null) {
            count++;
            temp = temp.next;
        }

        int middle = count / 2;

        temp = head;

        // Move to middle
        for(int i = 0; i < middle; i++) {
            temp = temp.next;
        }

        return temp;
    }

    public static void main(String[] args) {
```

```
Node head = new Node(10);
head.next = new Node(20);
head.next.next = new Node(30);
head.next.next.next = new Node(40);
head.next.next.next.next = new Node(50);

Node ans = middleNode(head);

System.out.println(ans.data);
}
}
```

Python Code (Brute Force)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
def middleNode(head):
```

```
    count = 0
```

```
    temp = head
```

```
    # Count nodes
```

```
    while temp:
```

```
        count += 1
```

```
        temp = temp.next
```

```
    middle = count // 2
```

```
    temp = head
```

```
    # Move to middle
```

```
    for i in range(middle):
```

```
        temp = temp.next
```

```
    return temp
```

```
head = Node(10)
```

```
head.next = Node(20)
```

```
head.next.next = Node(30)
```

```
head.next.next.next = Node(40)
```

```
head.next.next.next.next = Node(50)
```

```
ans = middleNode(head)
```

```
print(ans.data)
```

Approach 2 — Slow and Fast Pointer

MOST IMPORTANT LINKED LIST TECHNIQUE

This is one of the most famous linked list patterns.

Intuition

Take:

- Slow pointer → moves 1 step
- Fast pointer → moves 2 steps

When fast reaches end:

- Slow automatically reaches middle
-

Visualization

Input:

10 → 20 → 30 → 40 → 50

Initial:

slow = 10

fast = 10

Movement

Step 1

Slow moves 1 step:

20

Fast moves 2 steps:

30

Step 2

Slow:

30

Fast:

50

Step 3

Fast cannot move further.

Stop.

Answer:

slow = 30

Why Does This Work?

Because:

- Fast moves twice as fast
- When fast completes full list
- Slow completes half list

Half of list = middle

Algorithm

1. slow = head
 2. fast = head
 3. While fast != NULL and fast.next != NULL
 - slow = slow.next
 - fast = fast.next.next
 4. Return slow
-

Time Complexity

Operation Complexity

Traversal $O(N)$

Only one traversal.

Space Complexity

$O(1)$

Dry Run (Slow Fast Pointer)

Input:

10 → 20 → 30 → 40 → 50

Initial

Slow Fast

10 10

Iteration 1

Slow moves 1 step:

20

Fast moves 2 steps:

30

Slow Fast

20 30

Iteration 2

Slow:

30

Fast:

50

Slow Fast

30 50

Iteration 3

Fast.next = NULL

Loop stops.

Answer:

30

Dry Run for EVEN Nodes

Input:

10 → 20 → 30 → 40 → 50 → 60

Initial

Slow Fast

10 10

Iteration 1

Slow Fast

20 30

Iteration 2

Slow Fast

30 50

Iteration 3

Slow Fast

40 NULL

Answer:

40

Second middle node returned.

C++ Code (Slow Fast Pointer)

```
#include <iostream>
using namespace std;
```

```
class Node {
```

```

public:
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = NULL;
    }
};

Node* middleNode(Node* head) {

    Node* slow = head;
    Node* fast = head;

    while(fast != NULL && fast->next != NULL) {

        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}

int main() {

    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);
    head->next->next->next = new Node(40);
    head->next->next->next->next = new Node(50);

    Node* ans = middleNode(head);

    cout << ans->data;

    return 0;
}

```

Java Code (Slow Fast Pointer)

```

class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
    }
}

```

```

        this.next = null;
    }
}

public class Main {

    static Node middleNode(Node head) {

        Node slow = head;
        Node fast = head;

        while(fast != null && fast.next != null) {

            slow = slow.next;
            fast = fast.next.next;
        }

        return slow;
    }

    public static void main(String[] args) {

        Node head = new Node(10);
        head.next = new Node(20);
        head.next.next = new Node(30);
        head.next.next.next = new Node(40);
        head.next.next.next.next = new Node(50);

        Node ans = middleNode(head);

        System.out.println(ans.data);
    }
}

```

Python Code (Slow Fast Pointer)

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def middleNode(head):

    slow = head
    fast = head

```

```
while fast and fast.next:
```

```
    slow = slow.next
```

```
    fast = fast.next.next
```

```
return slow
```

```
head = Node(10)
```

```
head.next = Node(20)
```

```
head.next.next = Node(30)
```

```
head.next.next.next = Node(40)
```

```
head.next.next.next.next = Node(50)
```

```
ans = middleNode(head)
```

```
print(ans.data)
```

Difference Between Approaches

Feature	Brute Force	Slow Fast Pointer
Traversals	2	1
Complexity	$O(N)$	$O(N)$
Efficient	Less	More
Interview Preferred	No	Yes

Important Interview Point

The slow-fast pointer technique is used in:

- Middle of linked list
- Detect cycle
- Find cycle starting point
- Happy number
- Palindrome linked list

Very important pattern.

Key Observation

Fast moves double speed of slow

So when fast finishes:

- Slow reaches half
 - Half means middle
-

Common Mistakes

Mistake 1

Wrong loop:

```
while(fast != null)
```

Why wrong?

Because:

- fast.next.next may cause NULL error

Correct:

```
while(fast != null && fast.next != null)
```

Mistake 2

Using:

```
fast = fast.next;
```

Then both move same speed.

Middle not found.

Real World Analogy

Imagine:

- Slow = walking person
- Fast = running person

When runner reaches finish line:

- Walker reaches halfway point

That halfway point = middle node.